

Reviewer Pack — Minimal Rank of Configuration Space Metrics over Tseitin For...

Entry e2fdb76135d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 15:18:17 UTC

Minimal Rank of Configuration Space Metrics over Tseitin Formulas

Entry ID: e2fdb76135d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 15:18:17 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For any Tseitin formula with n variables, the minimal rank of its associated configuration space metric is at least $\Omega(n^2 \log n)$.’, ‘This implies that resolution refutations for Tseitin formulas require a length of at least $\Omega(n^2 \log n)$ if the metrics are considered.’, ‘Moreover, there exists a constructive mapping from the Tseitin formula to its configuration space metric such that this lower bound is achieved.’]

Rationale (proposer’s reasoning):

[‘Configuration spaces have been studied in geometry and topology, with potential applications in understanding complex systems. Their metrics could provide new insights into the structure of resolution proof trees for Tseitin formulas.’, ‘This conjecture links geometric properties with complexity theory, which has not been extensively explored before in this context.’, ‘If true, it would suggest that the difficulty of Tseitin formulas in resolution complexity is deeply rooted in their geometric nature.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b1c8a93ffec256c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Tseitin formula with n variables, the configuration space metric’s minimal rank will be considered supported if the mean rank across all 30 seeds is at least $\Omega(n^2 \log n)$ AND no seed shows a rank less than $n^2 \log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	SAFE
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - configuration space AND Tseitin formulas AND resolution proof complexity-metric rank AND $\Omega(n^2 \log n)$ AND configuration space-Tseitin formula mapping TO configuration space metric AND resolution refutations

Top relevant hits considered: - [http://arxiv.org/abs/2209.05839v3] On bounded depth proofs for Tseitin formulas on the grid; revisited - [http://arxiv.org/abs/1008.1789v1] Understanding Space in Proof Complexity: Separations and Trade-offs via Substitutions - [http://arxiv.org/abs/1003.3047v2] On the Relative Strength of Pebbling and Resolution - [http://arxiv.org/abs/2506.13081v1] The Combinatorial Rank of Subsets: Metric Density in Finite Hamming Spaces - [http://arxiv.org/abs/1311.4291v2] Minimum n -Rank Approximation via Iterative Hard Thresholding - [http://arxiv.org/abs/1508.06813v3] The moduli space of instantons on an ALE space from 3d $\mathcal{N} = 4$ field theories - [http://arxiv.org/abs/0804.4433v1] SPACE: the SPectroscopic All-sky Cosmic Explorer - [http://arxiv.org/abs/1910.01506v1] Whistler instability stimulated by the suprathermal electrons present in space plasmas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for var in variables:
            clauses.append([var])
        for i in range(1, n):
            clauses.append([f'~{variables[i-1]}', f'{variables[i]}'])
        return clauses

    def configuration_space_metric(clauses):
        points = set()
        distances = {}
        for clause in clauses:
            point = tuple(sorted(clause))
```

```

        if point not in points:
            points.add(point)
        for other_point in points:
            dist = sum(1 for a, b in zip(point, other_point) if a != b)
            distances[(point, other_point)] = dist
    return points, distances

def min_rank(points, distances):
    n = len(points)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for i, point1 in enumerate(points):
        for j, point2 in enumerate(points):
            if (point1, point2) in distances:
                adjacency_matrix[i][j] = distances[(point1, point2)]

    rank = 0
    for i in range(n):
        max_edge = 0
        for j in range(i+1, n):
            if adjacency_matrix[i][j] > max_edge:
                max_edge = adjacency_matrix[i][j]
        rank += max_edge
    return rank

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find the pivot row
        max_row = i
        for j in range(i+1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        for j in range(i+1, rows):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]

    rank = 0
    for row in matrix:
        if any(row):
            rank += 1
    return rank

def min_rank_gaussian(matrix):
    n = len(matrix)
    augmented_matrix = [row + [1] for row in matrix]
    rank = gaussian_elimination(augmented_matrix)
    return rank

def construct_tseitin_metric(n):
    clauses = tseitin_formula(n)
    points, distances = configuration_space_metric(clauses)
    matrix = [[0] * len(points) for _ in range(len(points))]

```

```

    for i, point1 in enumerate(points):
        for j, point2 in enumerate(points):
            if (point1, point2) in distances:
                matrix[i][j] = distances[(point1, point2)]
    return matrix

n_values = [5, 10, 15, 20, 30, 40]
ranks = []

for n in n_values:
    matrix = construct_tseitin_metric(n)
    rank = min_rank_gaussian(matrix)
    ranks.append(rank)

mean_rank = sum(ranks) / len(ranks)
conjecture_holds = all(rank >= n**2 * math.log(n) for rank, n in zip(ranks, n_values))

return {
    "metric_name": "min_rank",
    "metric_value": mean_rank,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"n={n}, rank={rank}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={0} support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={0} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbd4c729.py", line 122, in <module>

```

result = run_trial(seed)
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbd4c729.py", line 102, in run_trial
    rank = min_rank_gaussian(matrix)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbd4c729.py", line 84, in min_rank_gaussian
    rank = gaussian_elimination(augmented_matrix)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bbd4c729.py", line 71, in gaussian_elimination
    factor = matrix[j][i] / matrix[i][i]
    ~~~~~^~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not meet the pre-registered support condition for a 'SUPPORTED' verdict. | next: Re-run the test with proper error handling to ensure that it can complete without crashing and provide the necessary data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12223
2	preregistration	ollama_remote	glm4:latest	0	0	5890
3	novelty	ollama_remote	glm4:latest	0	0	4619
4	novelty	ollama_remote	glm4:latest	0	0	6294
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40332
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11758
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12197
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29497
9	judge	ollama_remote	glm4:latest	0	0	53102

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 175912 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e2fdb76135d2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/e2fdb76135d2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e2fdb76135d2.tar.gz` (if generated)